

Projet à faire

Consignes de remise du projet

- Chaque projet doit être accompagné d'un **rapport détaillé** et du **code source complet**.
- Le **rapport** et le **code source** doivent être envoyés ou déposés sur la plateforme **séparément** (deux fichiers distincts).
- **Deux étudiants maximum** sont autorisés par projet.
- **Un même projet ne peut pas être attribué à plusieurs groupes** : chaque projet est unique par groupe.
- Objet du mail : **PROJETWEBSERVICESFOAD2026**
- Mail : **papasamour.diop@ugb.edu.sn**

Date limite : 1 avril 2026

Projet 1 : "SenE-Pharmapay"

Ce projet fera dialoguer différents systèmes hétérogènes.

Le Scénario (Cas d'usage) :

Un patient souhaite commander des médicaments vitaux dans une pharmacie de garde. L'application mobile (Client) va interroger une API REST pour trouver la pharmacie, vérifier la couverture de la carte d'assurance maladie du patient via un service SOAP, initier un paiement Wave, et informer le patient par SMS que sa commande est prête à être récupérée.

Architecture et Répartition par Compétence (Chapitres du cours) :

- **Composant 1 : Le Service "Assurance / Banque Centrale" (Compétence Module 2 - SOAP & Java)**
 - **Technologie** : Java (JAX-WS, Maven).
 - **Rôle** : Vous allez créer un service SOAP (avec son fichier WSDL) nommé AssuranceSanteWS.
 - **Action** : Ce service exposera une méthode `verifierCouverture(String numeroCarte)` qui renverra le pourcentage de prise en charge du patient. Cela démontrera votre maîtrise de la rigueur XML et du concept "Contract-First".
- **Composant 2 : Le Cœur du Système (Compétence Module 3 - REST & Node.js)**
 - **Technologie** : Node.js avec Express.
 - **Rôle** : Création d'une API RESTful respectant le nommage strict des ressources.
 - **Endpoints** : * GET /pharmacies?ville=Dakar&garde=true (Pour trouver où acheter).

- POST /commandes (Pour initier une commande de médicament).
- **Composant 3 : La Sécurité et la Documentation (Compétence Module 4 - JWT & Swagger)**
 - **Swagger** : L'API REST Node.js sera entièrement documentée via OpenAPI/Swagger (/api-docs), permettant aux développeurs front-end de tester les routes.
 - **JWT** : L'accès pour modifier les stocks des pharmacies (PUT /pharmacies/:id/stock) sera protégé. Seul un pharmacien authentifié possèdera un jeton (Token) JWT valide pour exécuter cette requête.
 - **Rate Limiting** : Implémentation de limites (ex: 100 requêtes/minute) pour éviter qu'un robot ne sature le système de recherche de pharmacies.
- **Composant 4 : L'Orchestrateur et le Temps Réel (Compétence Module 5 - Webhooks, Wave, SMS & Mashup)**
 - **Webhook Wave** : Votre serveur Node.js possèdera une route POST /webhooks/wave qui écoutera les confirmations de paiement venant des serveurs de Wave.
 - **Sécurité HMAC** : Le webhook vérifiera la signature du paiement avec la clé secrète pour s'assurer que la requête vient bien de Wave et n'est pas falsifiée.
 - **Le Mashup (L'action finale)** : Une fois le paiement Wave validé par le Webhook, votre serveur met à jour le statut de la commande dans la base de données, et **appelle automatiquement l'API externe d'Orange** pour envoyer un SMS au patient : *"Votre paiement de 5000 FCFA est validé. Votre médicament vous attend à la Pharmacie de la Nation."*

Projet 2 : "Sunu-Transport API" (Billetterie Intelligente et Orchestration)

Le Scénario (Cas d'usage) :

Un étudiant habitant à Guédiawaye souhaite acheter un ticket de BRT avec un tarif subventionné (réduction étudiant). L'application mobile va d'abord lister les horaires (REST), vérifier le statut de l'étudiant auprès du Ministère de l'Enseignement Supérieur (SOAP), traiter le paiement via Orange Money (Webhook), et envoyer un e-ticket avec QR Code directement sur le téléphone du client par SMS ou WhatsApp (Mashup).

Architecture et Répartition par Compétence (Chapitres du cours) :

1. Le Service "État / Ministère" (Compétence Module 1 & 2 - HTTP, SOAP & Rigueur XML)

- **Le Rôle** : Les systèmes gouvernementaux sont souvent anciens et nécessitent une sécurité stricte. Vous allez simuler le système du Ministère ou de la Direction des Bourses.
- **Technologie** : Java avec JAX-WS.
- **Mise en œuvre** :
 - **Approche Contract-First** : Vous allez d'abord écrire le fichier WSDL (le "Notaire Numérique") définissant le service BourseWS.
 - **Méthode** : `verifierStatutEtudiant(String numeroCarteEtudiant)` qui retourne un fichier XML strict indiquant si l'étudiant est valide et son taux de réduction applicable (ex: 50%).

2. Le Cœur du Système de Transport (Compétence Module 3 - Architecture REST & Modélisation)

- **Le Rôle** : C'est le moteur principal qui gère les bus, les trajets et les tickets, conçu pour être léger et rapide (JSON).
- **Technologie** : Node.js & Express.
- **Mise en œuvre** :
 - **API-First** : Modélisation de l'API avant le code. L'architecture doit être strictement "Stateless".
 - **Ressources (Endpoints)** :
 - `GET /trajets?depart=Guediawaye&arrivee=Petersen&heure=08:00` (Pour chercher un bus).
 - `POST /billets` (Pour créer une intention d'achat de billet).

3. Sécurité des Contrôleurs et Documentation (Compétence Module 4 - JWT, Swagger & Rate Limiting)

- **Documentation (Swagger)** : Toute l'API Node.js sera documentée avec Swagger UI. Les développeurs de l'application mobile de Sunu-Transport utiliseront cette interface pour comprendre comment interagir avec l'API.
- **Sécurité Stateless (JWT)** : Les contrôleurs dans les bus auront une application pour scanner les tickets. Lorsqu'un contrôleur se connecte au matin, il reçoit un **Token JWT**. Ce token sera exigé pour accéder à la route critique : PUT /billets/{id}/valider.
- **Rate Limiting** : Pour empêcher un attaquant de bloquer toutes les places d'un bus en créant de fausses réservations en boucle, vous implémenterez une limitation stricte (ex: maximum 5 requêtes de réservation par minute par IP).

4. L'Orchestration et l'Événementiel (Compétence Module 5 - Webhooks, HMAC & Mashup)

- **Le Webhook de Paiement** : L'utilisateur paie son ticket via Orange Money. Orange Money va envoyer une requête vers votre serveur lorsque le client aura composé son code secret. Vous allez créer la route POST /webhooks/orangemoney pour "écouter" cet événement.
- **Sécurité HMAC ("Le cachet de la poste")** : Pour éviter que quelqu'un falsifie le webhook pour valider son ticket gratuitement, votre système va vérifier la signature cryptographique (HMAC) de la requête envoyée par l'opérateur.
- **Le Mashup (L'Agrégation)** : Une fois le paiement confirmé, votre API (le chef d'orchestre) va :
 1. Générer le billet dans sa propre base de données.
 2. Interroger une API externe génératrice de QR Code.
 3. Interroger l'API d'un fournisseur SMS (comme Twilio ou l'API SMS d'un opérateur local) pour envoyer le message : *"Trajet confirmé pour Petersen. Voici votre lien QR Code : [Lien]"*.

Projet 3 : "Sunu-AgriTrade"

Le Scénario (Cas d'usage) :

Un supermarché basé à Dakar souhaite acheter 2 tonnes de mil directement à une coopérative paysanne située à Kaolack. L'application web va chercher les coopératives disponibles (REST), vérifier auprès du Ministère de l'Agriculture si la coopérative possède la certification phytosanitaire officielle (SOAP), traiter le paiement instantané du supermarché vers la coopérative via Mobile Money (Webhook), et enfin déclencher l'envoi d'un SMS au chef du village pour confirmer que les camions peuvent venir charger la marchandise (Mashup).

Architecture et Répartition par Compétence (Chapitres du cours) :

1. Le Service "Ministère de l'Agriculture" (Compétences Modules 1 & 2 - Rigueur SOAP et XML)

- **Le Rôle** : Le gouvernement maintient un registre strict des agriculteurs certifiés. L'administration exige des standards rigoureux et sans ambiguïté.
- **Technologie** : Java avec JAX-WS pour créer un service SOAP.
- **Mise en œuvre** :
 - **Approche Contract-First** : Vous rédigerez d'abord le WSDL (le "Notaire Numérique") pour définir les règles d'interopérabilité avant même de coder.
 - **Méthode** : `verifierCertification(String identifiantCooperative)`. Cette méthode recevra et renverra du XML strict (validé par un XSD) pour s'assurer que le dossier administratif de la coopérative est conforme.

2. Le Cœur de la Bourse Agricole (Compétences Module 3 - REST, JSON et Modélisation)

- **Le Rôle** : Un système rapide et léger qui gère le catalogue des récoltes et les commandes, conçu de manière "Stateless" (sans état) pour supporter un trafic mondial.
- **Technologie** : Node.js & Express.
- **Mise en œuvre** :
 - **Utilisation du JSON** : Format natif et ultra-léger pour économiser la data des utilisateurs en zone rurale.
 - **Ressources (Endpoints respectant la sémantique HTTP)** :
 - `GET /cooperatives?region=Kaolack&produit=Mil` : Méthode *sûre* et *idempotente* pour consulter les stocks sans modifier la base de données.
 - `POST /commandes` : Pour créer une nouvelle transaction d'achat.

3. Sécurisation et Documentation de la Bourse (Compétences Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation avec Swagger** : L'API fournira une route `/api-docs` générant une interface visuelle ("le manuel qui s'écrit tout seul") permettant aux acheteurs de comprendre comment connecter leur propre système d'achat.
- **Sécurité JWT** : Seuls les gérants de coopératives posséderont un Token JWT. Ce badge virtuel, transmis dans le Header de la requête, sera obligatoire pour accéder à la route critique `PUT /cooperatives/{id}/stock` (mise à jour des tonnes disponibles après une récolte).
- **Rate Limiting** : Pour éviter que des concurrents ne "scrappent" (aspirent) tout le catalogue des prix agricoles, le serveur bloquera les adresses IP dépassant une certaine limite (ex: réponse HTTP 429 Too Many Requests au-delà de 100 requêtes/minute).

4. Logistique et "Chef d'Orchestre" (Compétences Module 5 - Webhooks, HMAC et Mashups)

- **Événement Asynchrone (Webhook)** : Une fois la commande passée, le paiement est externalisé vers un opérateur (ex: Free Money ou Wave). Votre serveur Node.js mettra à disposition une route `POST /webhooks/paiement` qui restera à l'écoute de la confirmation de l'opérateur.
- **Sécurité Cryptographique (HMAC)** : Pour empêcher un pirate de simuler une validation de paiement avec un simple JSON `{"status": "SUCCESS"}`, le Webhook vérifiera la signature de l'opérateur (l'empreinte numérique calculée avec une clé secrète partagée).
- **Le Mashup (Création de Valeur)** : Dès que le Webhook valide le paiement de manière sécurisée, votre serveur orchestre l'étape finale en appelant l'API SMS d'un opérateur télécom pour notifier le paysan : *"Paiement de 500 000 FCFA reçu de Supermarché X. Préparez 2 Tonnes de Mil."*

Projet 4 : "Sunu-Scolarité"

Le Scénario (Cas d'usage) :

Un nouveau bachelier souhaite s'inscrire en première année à l'université.

L'application portail va d'abord vérifier l'authenticité de son diplôme auprès de l'Office du Bac (via un service SOAP très strict). Ensuite, l'étudiant va choisir sa filière dans le catalogue de l'université (API REST). Il va payer ses droits d'inscription via un opérateur comme TouchPay ou Wave (Webhook). Enfin, le système va générer sa carte d'étudiant numérique et lui envoyer ses identifiants universitaires par SMS (Mashup).

Architecture et Répartition par Compétences (Chapitres du cours du Dr DIOP) :

1. Le Service "Office du Baccalauréat" (Compétences Modules 1 & 2 - SOAP, Java et Rigueur XML)

- **Le Rôle** : L'Office du Bac est une institution étatique. Son système d'information doit être inviolable, rigoureux, et basé sur des contrats stricts (le "Notaire Numérique").
- **Technologie** : Java avec JAX-WS.
- **Mise en œuvre** :
 - **Contract-First** : Vous commencerez par écrire le fichier WSDL qui définit exactement comment interroger le service d'authentification des diplômes.
 - **Méthode** : `verifierAuthenticiteBac(String numeroTable, int annee)`.
 - **Fonctionnement** : Cette méthode recevra une requête XML lourde mais sécurisée, et retournera un XML validé certifiant que l'étudiant a bien obtenu son diplôme avec la mention correspondante.

2. Le Cœur du Système Universitaire (Compétences Module 3 - Architecture REST & Modélisation)

- **Le Rôle** : Le système de l'université gère les filières, les départements et les dossiers des étudiants. Il doit supporter des milliers de connexions simultanées (lors de l'ouverture des inscriptions). Il sera donc léger (JSON) et *Stateless*.
- **Technologie** : Node.js & Express.
- **Mise en œuvre (Ressources HTTP)** :
 - `GET /filieres?departement=informatique` : Pour lister les offres de formation (utilisation des paramètres de requête).
 - `POST /inscriptions` : Pour soumettre un nouveau dossier d'inscription.

3. Sécurisation de l'Administration et Documentation (Compétences Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation (Swagger)** : L'API Node.js exposera une interface Swagger (/api-docs) pour que les développeurs web ou mobiles de l'université sachent exactement comment intégrer les différents *endpoints*.
- **Sécurité JWT** : Le personnel de la scolarité utilisera un tableau de bord. Une fois connectés, ils recevront un **Token JWT**. Ce token sera exigé dans le Header HTTP (Bearer) pour accéder à des routes sensibles comme PUT /etudiants/{id}/valider-dossier.
- **Rate Limiting** : Durant la période des inscriptions, pour éviter les attaques par déni de service (DDoS) ou les scripts automatisés qui satureraient les serveurs de l'université, vous mettrez en place un limiteur (ex: 50 requêtes par minute par IP).

4. Orchestration des Inscriptions (Compétences Module 5 - Webhooks, HMAC et Mashups)

- **Le Webhook de Paiement (L'Écouteur)** : Une fois le dossier soumis, l'étudiant paie ses 25 000 FCFA d'inscription via Wave ou TouchPay. L'opérateur enverra une notification asynchrone à votre serveur via une route POST /webhooks/paiement-scolarite.
- **La Signature HMAC ("Le cachet de la poste")** : Pour éviter qu'un étudiant en informatique ne simule une requête postman disant {"status": "SUCCESS", "montant": 25000} pour s'inscrire gratuitement, votre Webhook recalculera l'empreinte cryptographique de la requête avec la clé secrète partagée avec l'opérateur financier.
- **Le Mashup (L'Agrégation finale)** : Votre API se transforme en "Chef d'orchestre" :
 1. Elle valide le paiement grâce à la signature HMAC.
 2. Elle met à jour le statut de l'étudiant en "Inscrit" dans sa base de données.
 3. Elle appelle une API externe de messagerie (ex: API Orange SMS) pour envoyer : *"Félicitations, votre inscription est validée. Votre numéro de carte étudiant est le 20261234. Connectez-vous sur votre portail."*

Projet 5 : "Sunu-Énergie"

Le Scénario (Cas d'usage) :

Un père de famille rentre chez lui le soir et se rend compte que son compteur d'électricité prépayé va bientôt se couper. Il utilise une application mobile pour acheter du crédit. Le système va vérifier la validité et l'identité associée au compteur auprès du système central de la compagnie d'électricité (SOAP). Ensuite, il initie un paiement via Mobile Money (Webhook). Une fois le paiement validé de manière sécurisée, le système génère le code de recharge à 20 chiffres et l'envoie instantanément par SMS au client (Mashup).

Architecture et Répartition par Compétences (Chapitres du cours du Dr DIOP) :

1. Le Service "Compagnie Nationale d'Électricité" (Modules 1 & 2 - SOAP, Java et Rigueur XML)

- **Le Rôle** : Le système central de la compagnie d'électricité (legacy system) est ancien, lourd, mais extrêmement rigoureux et sécurisé. Il détient la base de données de tous les compteurs du pays.
- **Technologie** : Java avec JAX-WS.
- **Mise en œuvre** :
 - **Approche Contract-First** : Vous allez rédiger le WSDL (le fameux "Notaire Numérique") pour dicter les règles strictes d'échange.
 - **Méthode** : `verifierCompteur(String numeroCompteur)`. Le service SOAP recevra la requête en XML et retournera un XML strict contenant le nom du client (ex: "M. Ndiaye") et le statut du compteur (Actif/Inactif).

2. Le Cœur de la Plateforme de Vente (Module 3 - Architecture REST & Modélisation)

- **Le Rôle** : C'est le serveur moderne (middleware) qui fera le pont entre l'application mobile du client, les moyens de paiement et le vieux système SOAP de la compagnie. Il doit être ultra-rapide (JSON) et **Stateless** (sans état).
- **Technologie** : Node.js & Express.
- **Mise en œuvre (Ressources HTTP API-First)** :
 - `GET /compteurs/{numero}` : Méthode idempotente pour récupérer les infos du compteur avant de payer.
 - `POST /recharges` : Pour créer une nouvelle transaction d'achat d'électricité.

3. Sécurisation des Boutiques et Documentation (Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation avec Swagger** : L'API exposera la route `/api-docs` pour générer une interface visuelle complète. Cela permettra aux développeurs d'applications tierces (comme les banques) d'intégrer facilement la vente de crédit électrique.

- **Sécurité JWT** : Vous allez créer un accès spécial pour les revendeurs agréés (les gérants de boutiques multiservices). Une fois connectés, ils reçoivent un **Token JWT**. Ce jeton devra être placé dans le *Header* HTTP pour accéder à la route `POST /recharges/revendeur` qui leur accorde une commission de 2% sur la vente.
- **Rate Limiting** : Pour éviter qu'un hacker ne lance un script pour deviner des numéros de compteurs ou saturer le système central, vous limiterez les requêtes à 10 tentatives par minute par adresse IP.

4. Orchestration et Délivrance du Code (Module 5 - Webhooks, HMAC et Mashups)

- **Écoute du Paiement (Webhook)** : L'utilisateur paie ses 10 000 FCFA via Wave ou Orange Money. Votre serveur Node.js va exposer une route `POST /webhooks/paiement-energie` qui attendra passivement la notification de l'opérateur financier.
- **Sécurité Cryptographique (HMAC)** : Pour éviter qu'un fraudeur n'envoie manuellement un faux statut `SUCCESS` à votre Webhook pour recevoir un code d'électricité gratuit, votre serveur vérifiera la signature de la requête (l'empreinte numérique calculée avec la clé secrète).
- **Le Mashup (L'Agrégation finale)** : Votre serveur devient le "chef d'orchestre" :
 1. Il valide le paiement (HMAC).
 2. Il interroge un service interne pour générer le code STS (Standard Transfer Specification) à 20 chiffres (ex: 1234-5678-9012-3456-7890).
 3. Il appelle l'API d'un fournisseur SMS pour envoyer au client : *"Paiement de 10 000 FCFA validé. Voici votre code de recharge Woyofal : 1234-5678-9012-3456-7890."*

Projet 6 : "Sunu-Papier"

Le Scénario (Cas d'usage) :

Un citoyen sénégalais né à Thiès mais résidant à Dakar a besoin d'un extrait de naissance et de son casier judiciaire pour postuler à un emploi. Au lieu de se déplacer, il utilise le portail web "Sunu-Papier". Le système va d'abord interroger la base de données de la Police/DAF pour vérifier son identité nationale (SOAP). Ensuite, le citoyen formule sa demande (REST) et paie le timbre fiscal via une solution de Mobile Money comme Wave ou PayDunya (Webhook). Une fois le paiement certifié, la plateforme génère le document numérique sécurisé et envoie le lien de téléchargement au citoyen par SMS (Mashup).

Architecture et Répartition par Compétences (Chapitres du cours du Dr DIOP) :

1. Le Service "Ministère de l'Intérieur / DAF" (Modules 1 & 2 - SOAP, Java et Rigueur XML)

- **Le Rôle** : Le registre national des identités est le cœur de la sécurité de l'État. C'est un système ancien, robuste, qui ne tolère aucune ambiguïté. Il fait office de "Notaire Numérique".
- **Technologie** : Java avec JAX-WS.
- **Mise en œuvre** :
 - **Contract-First** : Vous concevez le fichier WSDL qui dicte comment les autres ministères peuvent interroger la base des cartes d'identité.
 - **Méthode** : `verifierIdentiteCitoyen(String numeroNIN, String dateDelivrance)`. La requête XML entrante est validée par un XSD strict, et la réponse XML confirme l'identité du citoyen pour éviter les fraudes.

2. Le Cœur du Portail "Sunu-Papier" (Module 3 - Architecture REST & Modélisation API-First)

- **Le Rôle** : C'est le serveur moderne qui gère le catalogue des documents disponibles, les mairies et les demandes en cours. Il est conçu pour supporter des milliers de requêtes simultanées de manière **Stateless**.
- **Technologie** : Node.js & Express.
- **Mise en œuvre (Ressources HTTP en JSON)** :
 - `GET /mairies/thies/documents?type=extrait-naissance` : Pour lister les documents disponibles et leurs tarifs (idempotent).
 - `POST /demandes` : Pour initialiser une nouvelle demande de document.

3. Sécurisation des Mairies et Documentation (Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation (Swagger UI)** : La route `/api-docs` affichera la documentation interactive générée par OpenAPI. Cela permettra aux développeurs des

différentes communes du Sénégal de connecter leurs propres logiciels internes au portail national.

- **Sécurité JWT** : Les Officiers d'État Civil des mairies disposeront d'un accès privilégié. Après authentification, ils recevront un **Token JWT** (signé avec HMAC). Ce jeton devra être présent dans le *Header HTTP (Authorization: Bearer)* pour valider manuellement une demande complexe via PUT `/demandes/{id}/approuver`.
- **Rate Limiting** : Pour empêcher un attaquant d'aspirer les données des citoyens ou de saturer les serveurs de l'État avec de fausses demandes, vous imposerez une limite (ex: 15 requêtes par minute par adresse IP).

4. Orchestration Financière et Notification (Module 5 - Webhooks, HMAC et Mashups)

- **Le Webhook du Timbre Fiscal** : Le citoyen paie son timbre (ex: 500 FCFA) avec Wave. Wave notifie votre serveur en arrière-plan via la route POST `/webhooks/paiement-timbre`.
- **La Signature HMAC ("Le cachet de la poste")** : C'est l'étape critique. Pour éviter qu'un petit malin ne contourne le paiement en envoyant une fausse requête `{"status": "SUCCESS"}` à votre serveur, ce dernier va recalculer l'empreinte cryptographique de la requête avec la clé secrète Wave. Si la signature correspond, le paiement est réel.
- **Le Mashup (L'Agrégation de Services)** : Dès que le webhook valide le paiement, votre serveur se mue en chef d'orchestre :
 1. Il met à jour le statut de la demande en "Payée".
 2. Il déclenche la génération du PDF de l'extrait de naissance (avec un QR Code de vérification).
 3. Il invoque l'API d'Orange (ou d'un autre agrégateur) pour envoyer le SMS final : *"Sunu-Papier : Votre paiement de 500 FCFA est validé. Votre extrait de naissance est disponible ici : [Lien sécurisé]"*.

Projet 7 : "Sunu-Billet"

Le Scénario (Cas d'usage) :

Un amateur de lutte souhaite acheter un billet VIP pour un grand combat à l'Arène Nationale. Le système doit d'abord s'assurer que le promoteur de l'événement est bien en règle auprès du CNG (Comité National de Gestion de la Lutte) via un service SOAP. Ensuite, le fan consulte les places disponibles (REST) et achète son billet via son portefeuille électronique (Wave ou Orange Money). Le système écoute la confirmation de paiement (Webhook), sécurise la transaction, génère un QR Code unique, et l'envoie par SMS au fan pour qu'il puisse le présenter à l'entrée du stade (Mashup).

Architecture et Répartition par Compétences (Chapitres du cours du Dr DIOP) :

1. Le Service "Autorité de Régulation / CNG" (Modules 1 & 2 - SOAP, Java et Rigueur XML)

- **Le Rôle :** Le CNG (ou le Ministère de la Culture) gère les licences des promoteurs. C'est une institution qui exige des échanges formels, traçables et stricts (le fameux "Notaire Numérique").
- **Technologie :** Java avec JAX-WS.
- **Mise en œuvre :**
 - **Approche Contract-First :** Vous allez rédiger le fichier WSDL qui définit les règles d'interrogation du registre des promoteurs.
 - **Méthode :** `verifierLicencePromoteur(String numeroLicence, String annee)`.
 - **Fonctionnement :** La requête entrante sera un document XML lourd mais hyper-structuré. Le serveur validera la demande et répondra par un XML confirmant que le promoteur a le droit d'organiser le combat, évitant ainsi les arnaques aux faux événements.

2. Le Cœur de la Billetterie (Module 3 - Architecture REST & Modélisation API-First)

- **Le Rôle :** C'est le moteur de la plateforme de vente. Lors de l'annonce d'un grand combat, le serveur va subir un pic de trafic massif. Il doit donc être très léger (JSON) et strictement **Stateless** (pour pouvoir multiplier les serveurs facilement).
- **Technologie :** Node.js & Express.
- **Mise en œuvre (Ressources HTTP) :**
 - `GET /evenements/combat-choc/billets?zone=VIP` : Une méthode *sûre* et *idempotente* pour consulter les places restantes.
 - `POST /reservations` : Pour bloquer une place et initier l'intention d'achat.

3. Sécurisation des Portes du Stade et Documentation (Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation (Swagger UI)** : L'API exposera une route `/api-docs` pour générer une documentation interactive. Cela permettra aux partenaires (ex: agences de voyage ou hôtels) d'intégrer la vente de billets directement dans leurs propres applications.
- **Sécurité JWT** : Le jour de l'événement, les contrôleurs aux portes de l'Arène Nationale utiliseront des terminaux mobiles. En se connectant au matin, ils recevront un **Token JWT**. Ce jeton, placé dans le *Header HTTP (Bearer)*, sera obligatoire pour appeler la route critique `PUT /billets/{id}/scanner`. Sans ce token, impossible de valider une entrée.
- **Rate Limiting (Anti-Marché Noir)** : Pour empêcher les revendeurs (scalpers) d'utiliser des robots pour acheter tous les billets en 2 secondes, vous limiterez strictement les requêtes sur la route d'achat (ex: maximum 4 billets par adresse IP et par minute).

4. Orchestration Financière et Délivrance (Module 5 - Webhooks, HMAC et Mashups)

- **L'Écoute du Paiement (Webhook)** : Le fan valide son paiement de 15 000 FCFA sur son application Wave. Wave envoie une notification asynchrone à votre serveur via la route `POST /webhooks/paiement-billet`.
- **La Signature HMAC ("Le cachet de la poste")** : C'est l'étape anti-fraude absolue. Pour éviter qu'un hacker n'envoie une fausse requête `{"status": "SUCCESS"}` à votre Webhook pour obtenir un billet VIP gratuit, votre serveur va recalculer l'empreinte cryptographique de la requête en utilisant la clé secrète fournie par Wave. Si les empreintes correspondent, le paiement est authentique.
- **Le Mashup (L'Agrégation finale)** : Une fois l'authenticité prouvée, votre serveur devient le chef d'orchestre :
 1. Il marque le billet comme "Vendu" dans sa propre base.
 2. Il appelle une API externe pour générer l'image du QR Code sécurisé.
 3. Il invoque l'API d'un agrégateur télécom (ex: Orange) pour envoyer le message : *"Sunu-Billet: Paiement validé. Voici votre accès VIP pour le combat de dimanche : [Lien vers le QR Code]"*.

Projet 8 : "Sen-Transit API"

Le Scénario (Cas d'usage) :

Un commerçant (ou un transitaire) a un conteneur de marchandises qui vient d'arriver au Port de Dakar. Avant de pouvoir envoyer un camion le récupérer, il doit s'acquitter des frais portuaires. L'application web "Sen-Transit" va d'abord interroger le système très strict de la **Douane Sénégalaise** (via SOAP) pour vérifier si le conteneur a bien payé ses taxes douanières. Ensuite, le transitaire consulte sa facture de manutention portuaire (REST) et la paie via une banque partenaire ou Mobile Money (Webhook). Une fois le paiement cryptographiquement validé, le système génère un "Bon à Enlever" électronique et envoie un SMS au chauffeur du camion avec le numéro de quai (Mashup).

Architecture et Répartition par Compétences (Chapitres du cours du Dr DIOP) :

1. Le Service "Système Informatique de la Douane" (Modules 1 & 2 - SOAP, Java et Rigueur XML)

- **Le Rôle** : La Douane est une institution régalienne. Son système d'information (comme GAINDE au Sénégal) exige une rigueur absolue, une traçabilité parfaite et l'utilisation de contrats stricts (le "Notaire Numérique").
- **Technologie** : Création d'un web service en Java avec JAX-WS.
- **Mise en œuvre** :
 - **Contract-First** : Vous allez créer un fichier WSDL qui définit comment les acteurs portuaires peuvent interroger le statut d'un conteneur.
 - **Méthode** : `verifierQuitusDouane(String numeroConteneur)`. Le port enverra une requête XML lourde, et la Douane répondra par un XML certifiant que la marchandise est "Libre à la consommation" ou "Bloquée".

2. Le Cœur du Système Portuaire (Module 3 - Architecture REST & Modélisation API-First)

- **Le Rôle** : C'est le serveur moderne qui gère le trafic des camions, l'emplacement des conteneurs sur les terminaux et la facturation. Il doit être **Stateless** (pour gérer le flux constant d'utilisateurs) et utiliser le JSON pour la rapidité.
- **Technologie** : Node.js & Express.
- **Mise en œuvre (Ressources HTTP)** :
 - `GET /conteneurs?statut=disponible&terminal=DPWorld` : Pour chercher les conteneurs prêts à être récupérés.
 - `POST /demandes-enlevement` : Pour créer une intention de venir chercher la marchandise avec un camion spécifique.

3. Sécurisation des Transitaires et Documentation (Module 4 - JWT, Swagger et Rate Limiting)

- **Documentation (Swagger UI) :** La route `/api-docs` affichera la documentation générée par OpenAPI. Cela permettra aux grandes entreprises de logistique de connecter leurs propres logiciels ERP directement à votre API portuaire.
- **Sécurité JWT :** Les agents transitaires agréés auront un compte. Lors de leur connexion, l'API leur délivrera un **Token JWT**. Ce badge d'accès numérique devra être glissé dans le *Header HTTP (Bearer)* pour avoir le droit d'appeler des routes critiques comme `PUT /conteneurs/{id}/assigner-camion`.
- **Rate Limiting :** Pour empêcher un robot d'aspirer la base de données pour savoir quels commerçants importent quoi (espionnage industriel), vous limiterez les requêtes (ex: 30 requêtes par minute par transitaire).

4. Orchestration Financière et Logistique (Module 5 - Webhooks, HMAC et Mashups)

- **L'Écoute du Paiement (Webhook) :** Le transitaire paie les frais de magasinage (ex: 150 000 FCFA) via un portail bancaire ou Wave. L'opérateur financier envoie une confirmation asynchrone à votre API sur la route `POST /webhooks/paiement-port`.
- **La Signature HMAC ("Le cachet de la poste") :** Pour éviter qu'un hacker fraudeur n'envoie un faux JSON `{"status": "SUCCESS"}` pour sortir un conteneur gratuitement, votre webhook va recalculer l'empreinte cryptographique du message avec la clé secrète.
- **Le Mashup (L'Agrégation de Services) :** C'est la consécration du projet. Une fois le webhook validé par HMAC, votre API "chef d'orchestre" va :
 1. Changer le statut du conteneur en "Bon à Sortir".
 2. Informer le système des barrières de sécurité du port.
 3. Invoquer l'API d'un fournisseur télécom (ex: Orange) pour envoyer au chauffeur du camion : *"Sen-Transit: Bon à enlever validé pour le conteneur MSKU123456. Présentez-vous à la porte 4 du Terminal Nord."*